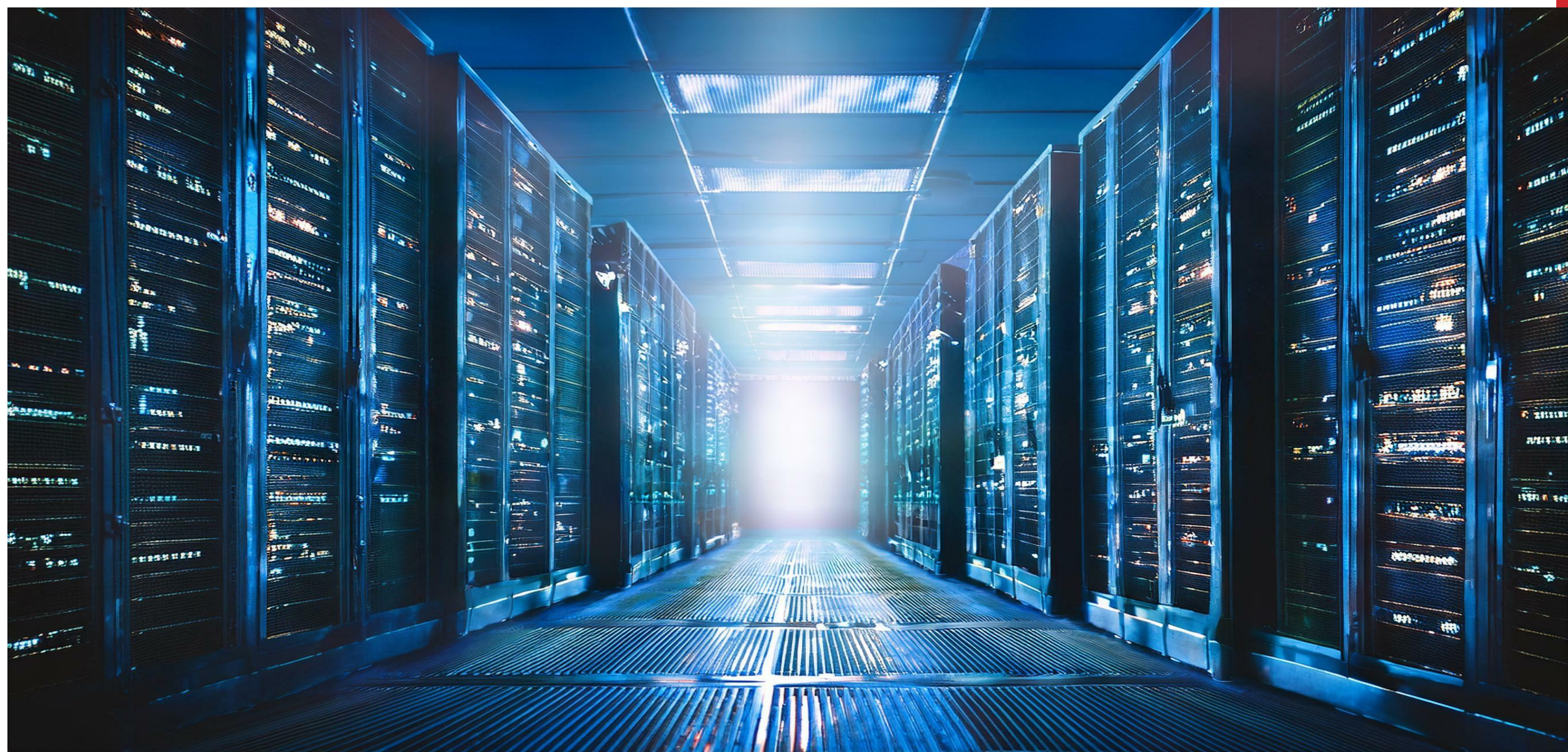


Kubernetes Integration & Modern Applications



LOGISTICS

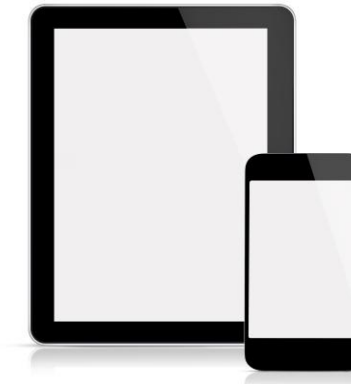
Class Hours:

- Class starts at X:00 AM/PM Time and Finish at XX:00 AM/PM
- There will be regular breaks in class.



Telecommunication:

- Turn off or set electronic devices to silent (not vibrate)
- Reading or attending to devices can be distracting to other students
- Try to delay until breaks or after class



COURSE STRUCTURE

Day 1: Storage Fundamentals & Architecture

Day 2: Flash Storage & Pure Storage

Day 3: Data Protection & Management

Day 4: Kubernetes Integration & Modern Applications

PREREQUISITES & EXPECTATIONS

- Basic understanding of computer systems, networking, and programming fundamentals
- One or more years of technical experience

AGENDA FOR DAY 4

1. Containers & Storage
2. Kubernetes CSI
3. PureStorage & Advanced CSI Feature



CONTAINERS & STORAGE

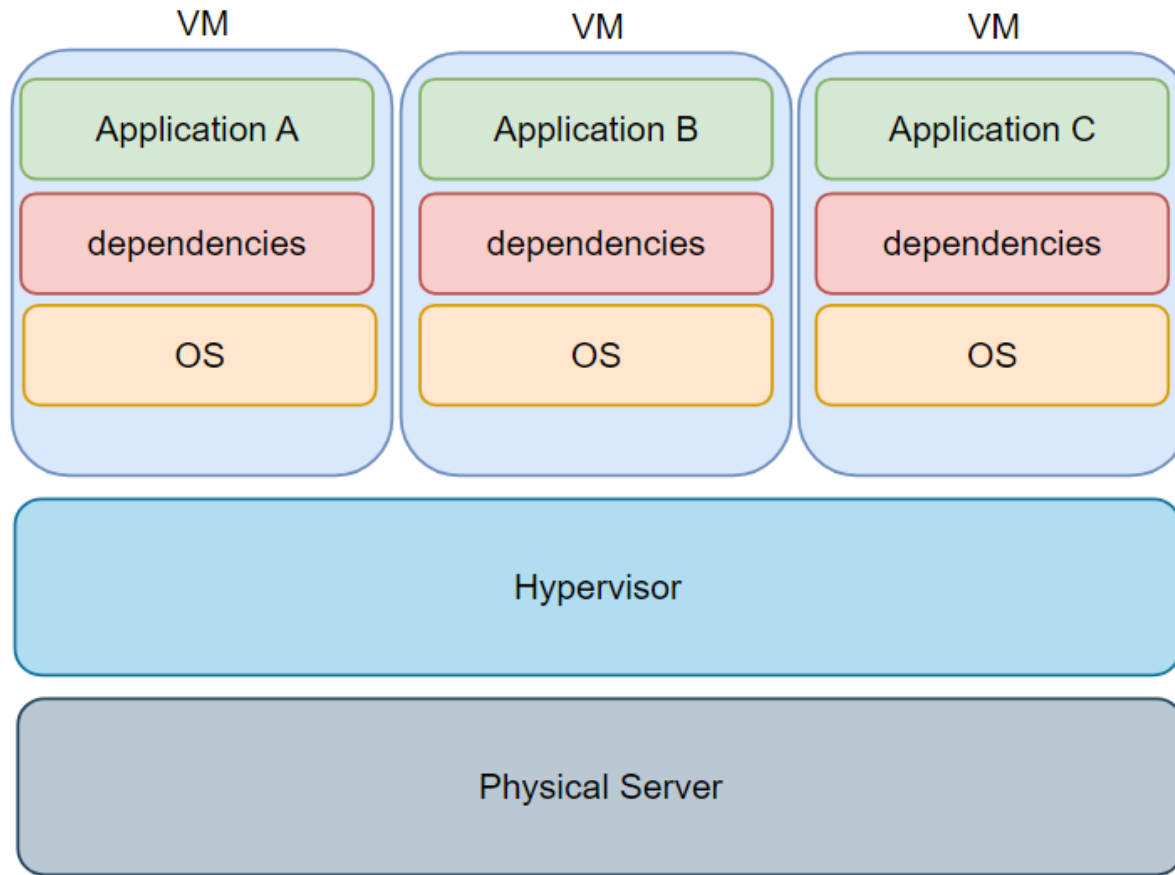
WHAT IS A CONTAINER?

- A container is a lightweight, portable unit that packages an application and its dependencies.
- Runs on a shared operating system kernel, isolated from other containers.
- Enables consistent environments across development and production.

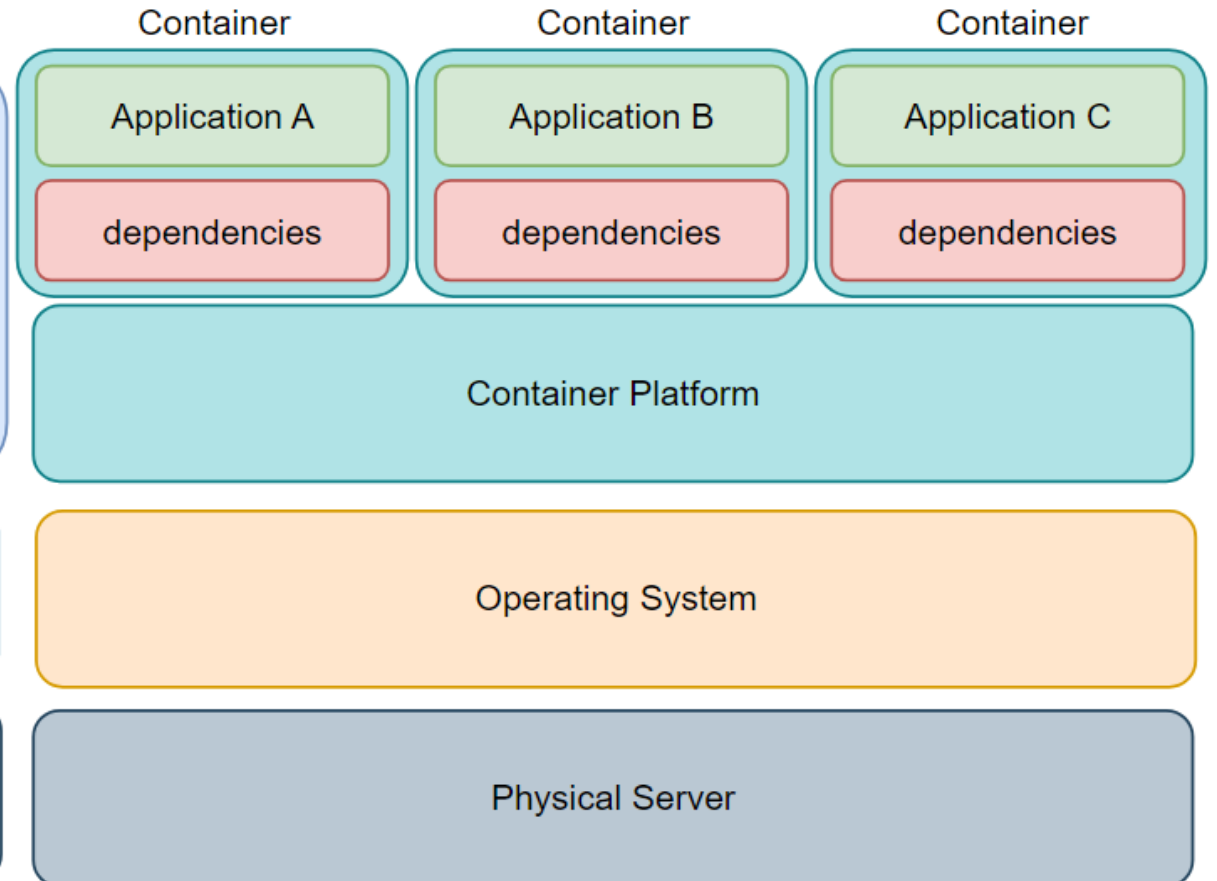


CONTAINER VS VIRTUAL MACHINES

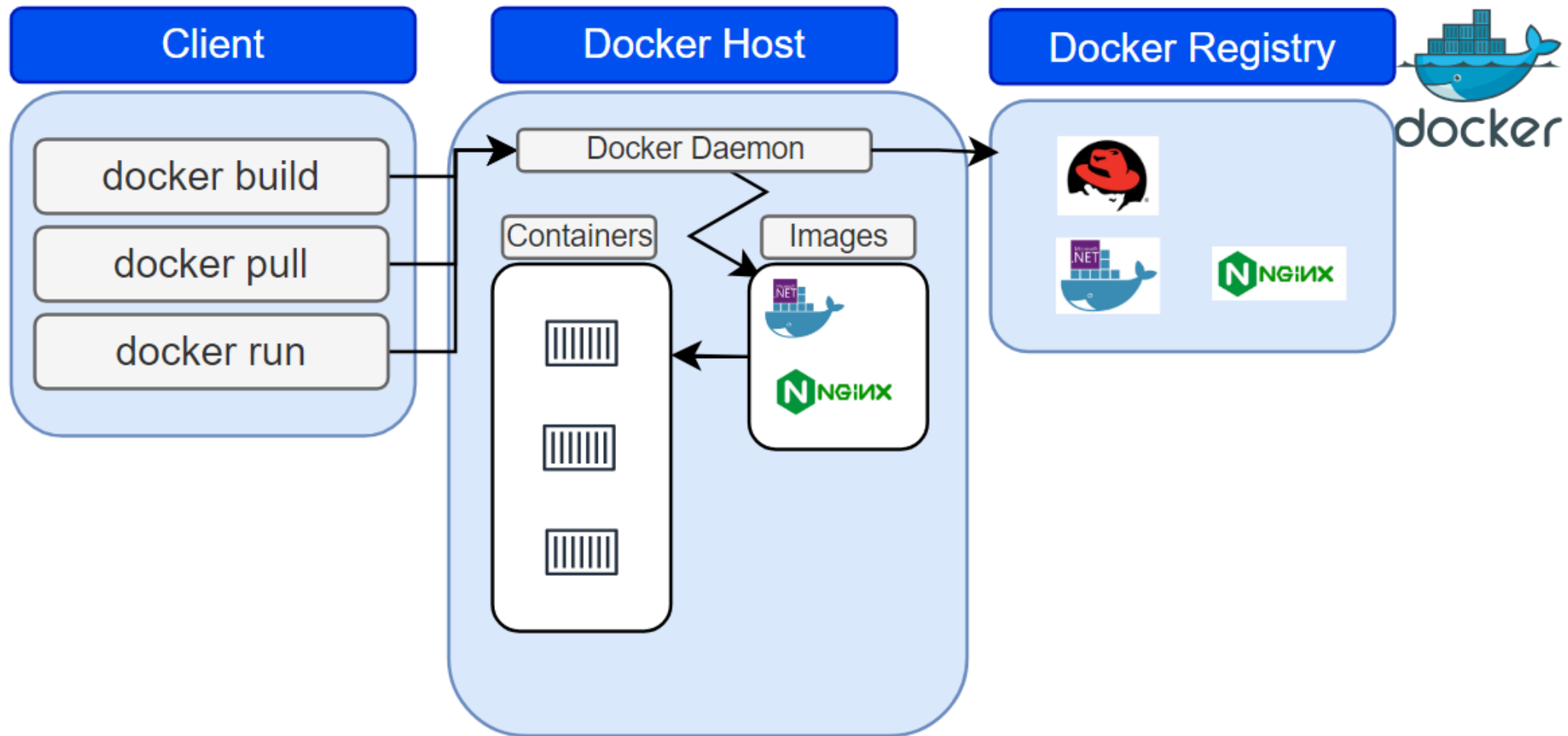
VM Architecture



Micro Services Architecture



CONTAINER LIFECYCLE



KUBERNETES CSI

TRADITIONAL VS. CONTAINER STORAGE

Feature	Traditional Storage Model	Container Storage Requirements
Provisioning	Static; manual LUN/volume assignment by administrators.	Dynamic, on-demand; developer self-service.
Configuration	Application-specific; often rigid.	Portable across infrastructure; automated lifecycle.
Scale	Hundreds of VMs; fewer, larger storage units.	Thousands of containers; many smaller, transient units.
Lifecycle	Persistent storage for persistent VMs.	Ephemeral by nature; storage must support rapid change.
Management	Manual, administrator-driven.	Automated, API-driven.

THE "STATEFUL" PROBLEM

Stateless

Pods are ephemeral : They can be terminated, rescheduled, or scaled at any time. This is perfect for web servers or API gateways.
Problem: If a pod writes data to its local filesystem, the data is *lost* when the pod is destroyed.

Stateful

Databases, message queues, and key-value stores *must* persist data. Their data lifecycle is independent of the pod lifecycle.
Challenge: How do we reliably attach, detach, and manage 'un-killable' storage disks for 'killable' pods?

THE SOLUTION: CONTAINER STORAGE INTERFACE (CSI)

"The Container Storage Interface (CSI) is a standard for exposing arbitrary block and file storage systems to containerized workloads."

- Kubernetes.io

CONTAINER STORAGE INTERFACE (CSI)

In a nutshell:

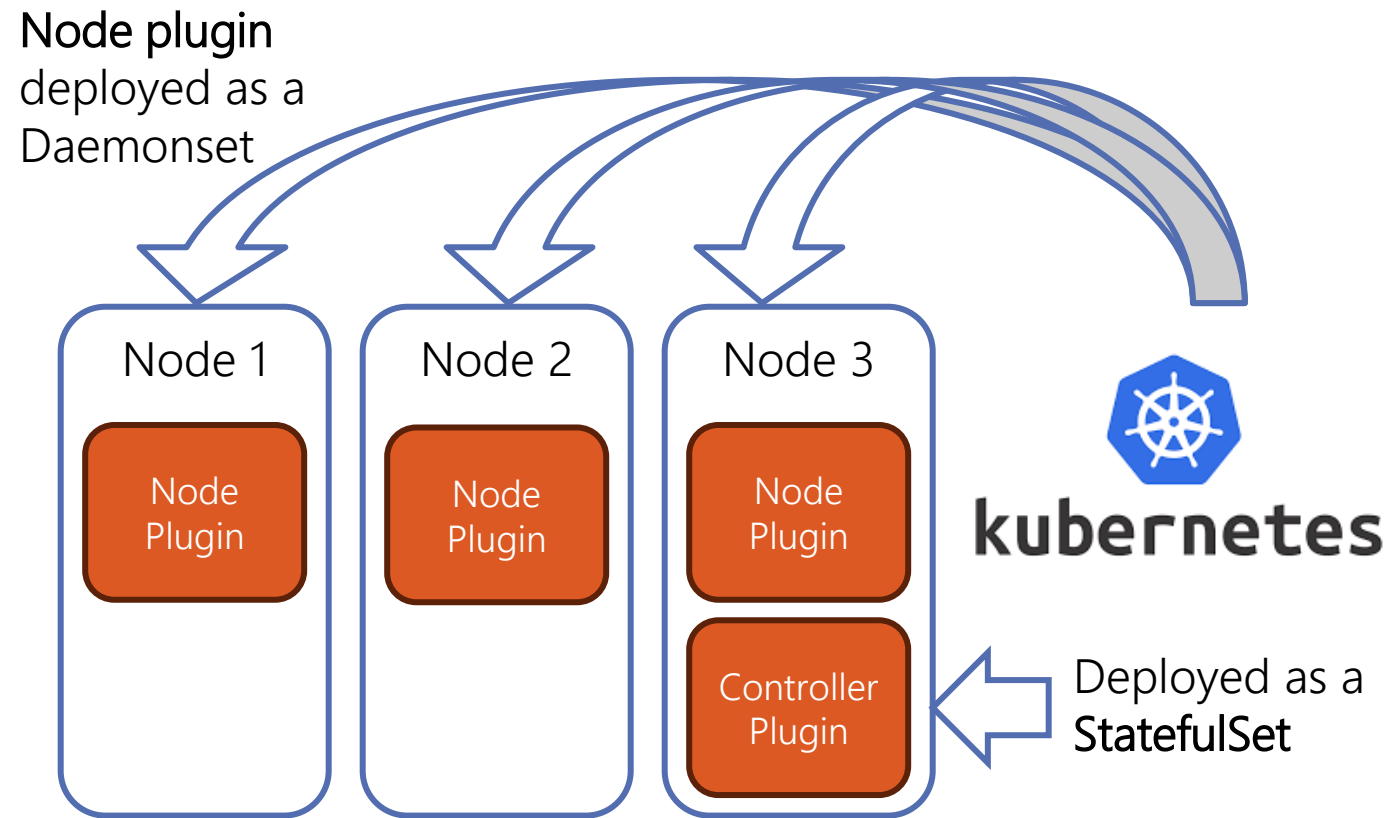
- Industry standard for storage integration
- Vendor-neutral interface specification
- Replaces in-tree Kubernetes storage plugins



Benefits:

- Vendor independence and choice
- Faster innovation and feature delivery
- Simplified Kubernetes storage development

CSI HIGH-LEVEL ARCHITECTURE



CSI decouples storage from Kubernetes. K8s provides the API, and vendors provide the 'driver.'

Drivers have two main components:

Controller Plugin

Central service that manages volumes
One per K8 cluster

Node Plugin

Service running on every single node

KUBERNETES STORAGE API: 3 PILLARS

StorageClass

The "Menu"
Administrator-
defined object that
describes the types
of storage offered

PersistentVolumeClaim

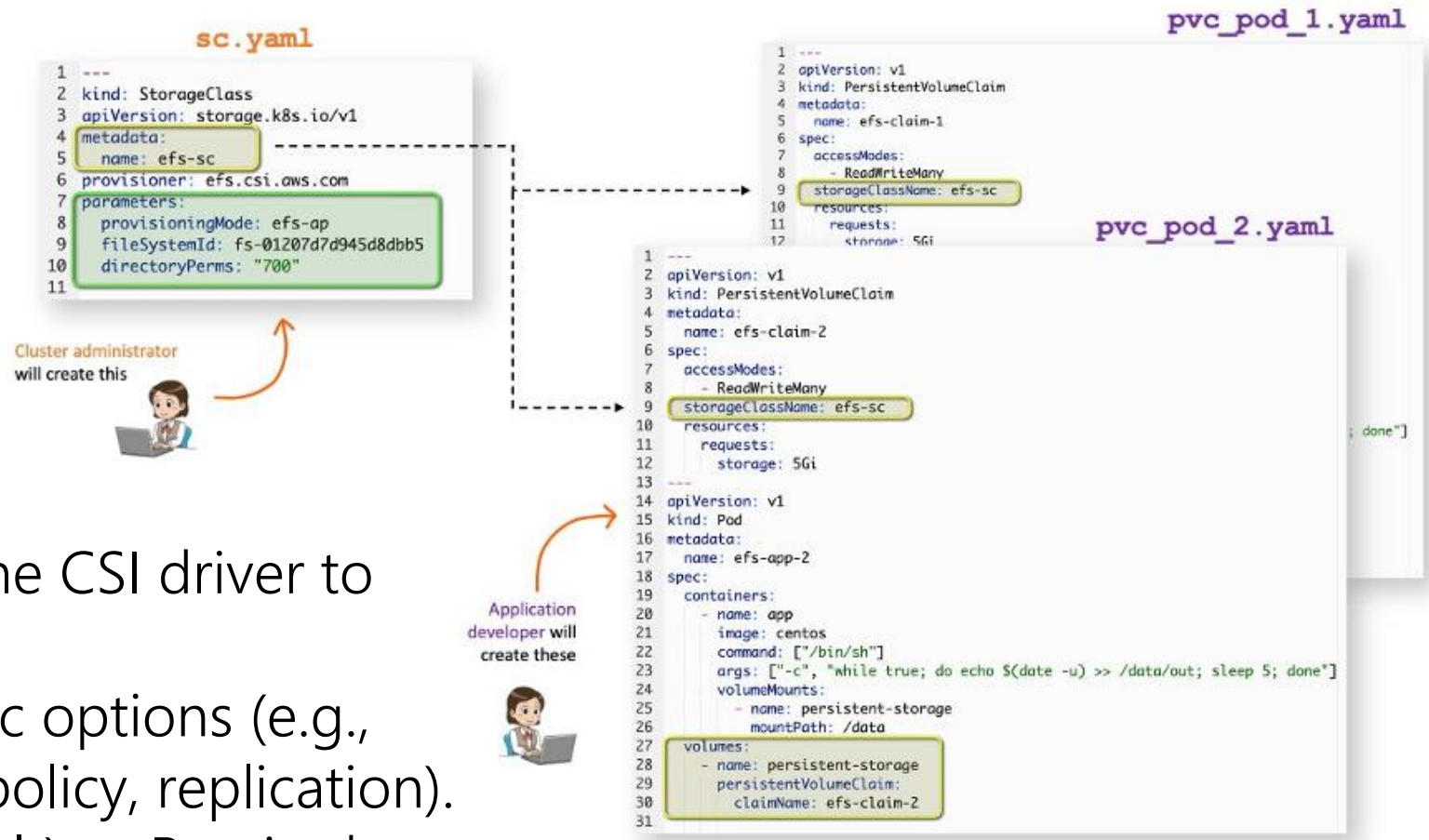
The "Request"
Developer's request
for storage.

PersistentVolume

The "Resource"
The actual
provisioned volume
in the cluster, bound
to a specific PVC

PILLAR 1: STORAGECLASS – THE MENU

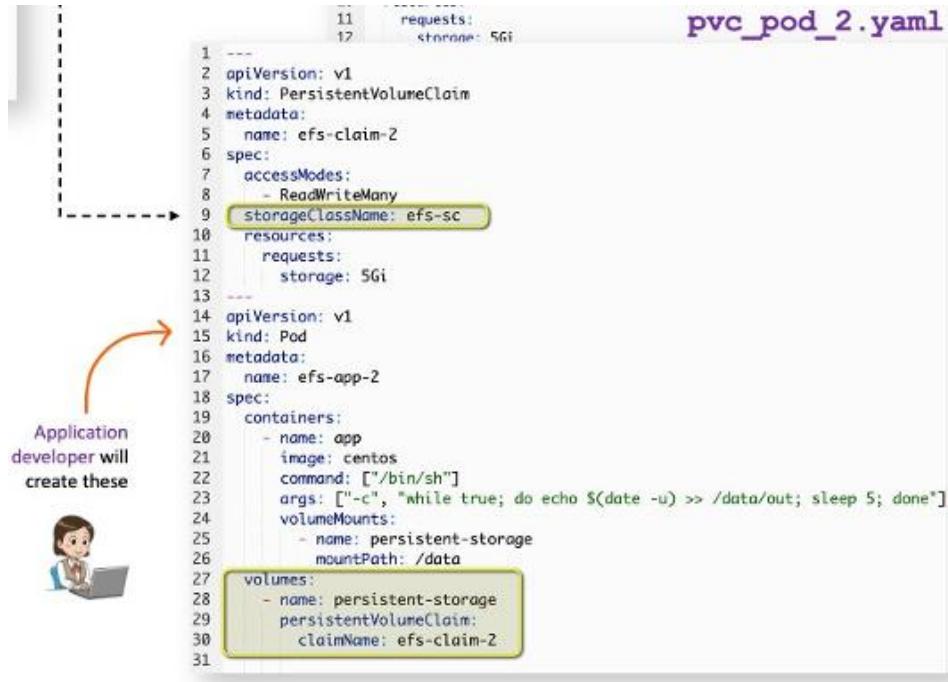
Template use to build a persistent volume with a concrete CSI driver and its parameters.



- **provisioner:** The name of the CSI driver to use. This is the critical link.
- **parameters:** Vendor-specific options (e.g., performance tier, storage policy, replication).
- **reclaimPolicy:** Delete (default) or Retain the volume when the user's claim is deleted.

PILLAR 2: PERSISTENTVOLUMECLAIM - THE REQUEST

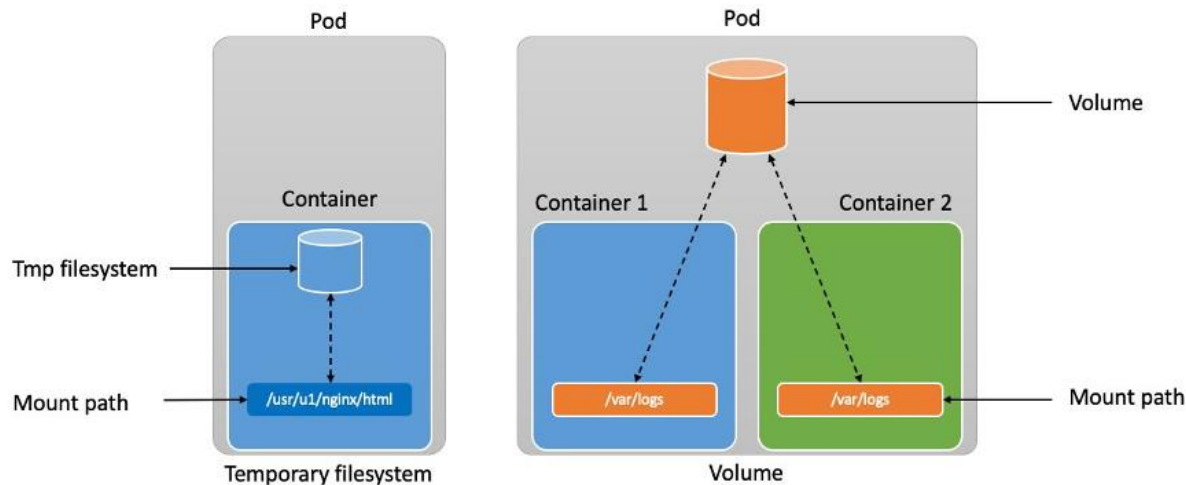
This is the 'Developer' object. It's an abstract request for storage, without needing to know how it's provisioned.



- **storageClassName:** The 'menu item' this request is for (e.g., fast-ssd).
- **accessModes:** How the volume can be mounted.
- **RWO (ReadWriteOnce):** Mount as read-write by a single node. (For databases).
- **RWX (ReadWriteMany):** Mount as read-write by many nodes. (For shared file systems).
- **resources.requests.storage:** How much storage is needed."

PILLAR 3: PERSISTENTVOLUME - THE RESOURCE

This object represents the actual provisioned volume. It is typically created automatically by the CSI driver (Dynamic Provisioning).



- **csi.volumeHandle**: The unique ID of this volume on the actual storage array. This is the link to the physical world.
- **capacity**: The true size of the provisioned disk.
- **claimRef**: A 'bind' that links this PV to the PVC that requested it. They are a 1-to-1 pair."

CSI VOLUME LIFECYCLE



Volume Creation

- Application creates PVC
- CSI Controller provisions volume on storage array
- Kubernetes creates PV object
- PVC binds to PV



Volume Attachment

- Pod scheduled to node
- CSI Controller attaches volume to node
- CSI Node plugin mounts volume in pod



Volume Cleanup

- Pod deleted, volume unmounted
- CSI Controller detaches volume
- PVC deleted, volume deprovisioned

POP QUIZ:

What are the key advantages of running containers ?

- A. Consistency
- B. Complexity
- C. Isolation
- D. Storage Simplification



POP QUIZ:

What are the key advantages of running containers ?

- A. Consistency
- B. Complexity
- C. Isolation
- D. Storage Simplification



POP QUIZ:

Container Storage Interface aims at provide specific configuration for each storage vendor. True or False ?



PURESTORAGE AND CSI ADVANCED FEATURE

PURE STORAGE CSI DRIVER

- Pure CSI Features:
 - Support for FlashArray and FlashBlade
 - Dynamic provisioning and volume expansion
 - Snapshots, clones, and data protection
 - Performance monitoring and analytics
- Supported Kubernetes Distributions:
 - Vanilla Kubernetes, OpenShift, Rancher
 - Amazon EKS, Azure AKS, Google GKE
 - VMware Tanzu, Anthos, Docker Enterprise
- Integration Capabilities:
 - Helm chart deployment
 - Operator-based management
 - Pure1 integration for monitoring

PURE CSI INSTALLATION & CONFIGURATION

- Prerequisites:
 - Kubernetes cluster (1.17+)
 - Pure Storage arrays with Purity 5.2+
 - Network connectivity and credentials
- Installation Methods:
 - Helm Chart: Recommended for production
 - kubectl apply: Direct YAML manifests
 - Operator: GitOps-friendly management
- Configuration:
 - Array connection details
 - Storage class definitions
 - Feature flags and parameters

ADVANCED CSI FEATURES

Volume Snapshots



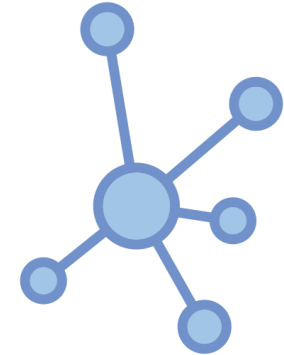
CSI snapshot CRDs and controller
Application-consistent snapshots
Clone from snapshot capability

Volume Expansion



Online volume expansion
File system resize
automation
Application awareness and
coordination

Topology and Node Affinity



Zone-aware provisioning
Node selector constraints
Performance optimization

MONITORING & OBSERVABILITY

- Metrics Collection:
 - CSI driver metrics (Prometheus compatible)
 - Volume performance statistics
 - Resource utilization tracking
- Logging Integration:
 - Structured logging format
 - Integration with log aggregation systems
 - Troubleshooting and debugging capabilities
- Pure1 Integration:
 - Kubernetes workload visibility
 - Performance analytics and optimization
 - Proactive support and recommendations

SECURITY & COMPLIANCE

- Storage Security:
 - Encryption at rest and in transit
 - RBAC integration with Kubernetes
 - Network policies for storage access
- Compliance Considerations:
 - Data sovereignty and residency
 - Audit trails and access logging
 - Backup retention and immutability
- Multi-tenancy:
 - Namespace isolation
 - Resource quotas and limits
 - Storage class governance

PERFORMANCE OPTIMIZATION

- Storage Performance Tuning:
 - I/O queue depths and parallelism
 - Block sizes and alignment
 - Node-level optimizations
- Application Optimization:
 - Database configuration for flash storage
 - Caching strategies and patterns
 - Connection pooling and batching
- Monitoring and Troubleshooting:
 - Performance bottleneck identification
 - Resource utilization analysis
 - Capacity planning and forecasting

FUTURE TRENDS & ROADMAP

- Container Storage Trends:
 - WebAssembly (WASM) and lightweight containers
 - Edge computing and distributed storage
 - Serverless and function-based architectures
- Kubernetes Evolution:
 - CSI specification enhancements
 - Storage capacity tracking and management
 - Cross-cluster storage federation
- Pure Storage Innovations:
 - Enhanced cloud-native integrations
 - AI/ML workload optimizations
 - Sustainability and energy efficiency

POP QUIZ:

PureStorage CSI driver is support by which Cloud provider?

- A. AWS
- B. Azure
- C. GCP
- D. All the above



POP QUIZ:

PureStorage CSI driver is support by which Cloud provider?

- A. AWS
- B. Azure
- C. GCP
- D. All the above



LAB 6 : DOCKER ON AWS EC2

- Install and configure Docker on Amazon Linux.
- Create a custom Bridge Network.
- Run a Redis container detached from the host.
- Build a custom Python web container.
- Connect containers manually so the Web App can talk to the Database.

LAB 7 : DEPLOYING ON KUBERNETES WITH S3 STORAGE

- Install Minikube (Single-node Kubernetes) on Amazon Linux.
- Create an AWS S3 Bucket and IAM Credentials.
- Install a CSI S3 Driver to allow Kubernetes to talk to S3.
- Deploy Redis and the Python App using Kubernetes Manifests.
- Verify that files written by the app appear in your S3 Bucket.